GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M2: Network and Synchronization

Instructor: Erkan Zergeroğlu **Company:** Board&Byte **Group:** Group-8 **Date:** 6 November 2026

Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Ayşe Poyraz	230104004085
Hasan Hüseyin Özdemir	230104004048	Umutcan Ocak	220104004903

1. Purpose

M2 connects the phones, the admin page and the LED board to the game server. It does not make game decisions. It takes messages from outside, checks them and gives them to M1. After each tick it takes the state from M1 and sends it to everybody. M2 also makes sure that nothing from outside can change the game state directly. For the admin, it measures the network and it can add delay and packet loss on purpose.

Runs on	Main computer, in the same server program as M1
Technology	TypeScript on Node.js; WebSocket for phones and admin; TCP and UDP for the LED board; MessagePack encoding; mDNS announcement; QR code for joining
Requirements	R1 R9 R10 R11

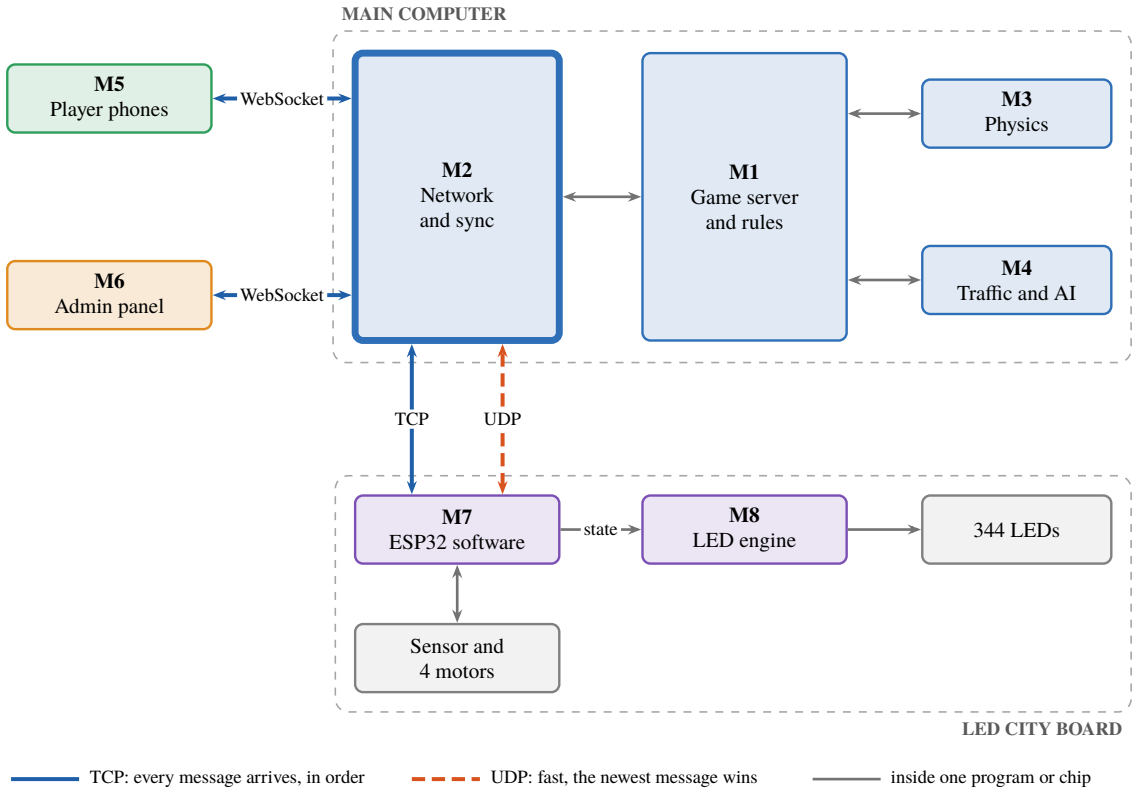

Figure 1: Where M2 sits in the system. All traffic from outside passes through M2.

2. Requirements covered

ID	Requirement	What M2 does for it
R1	Multiplayer game	Up to 8 phones join with a QR code or a 6-digit game code, only with the browser
R9	Live admin panel	Admin channel for commands, parameters, state, metrics and board data
R10	Network and measurement	Ping, packet loss and frame rate for each player; added delay and loss for each player and for the board

ID	Requirement	What M2 does for it
R11	Single authority	Inputs are checked and queued; every phone gets the same frozen state of each tick

3. Inputs and outputs

Dir.	Data	From / to	Content
In	Phone messages	M5	Hello with player key, name and game code; join with team; controls; “Find me”; ping replies
In	Admin messages	M6	Login, commands, parameter changes, requests for parameters and the game log
In	Board messages	M7	Earthquake events, sensor samples, board status, delay replies
In	State and metrics	M1	Frozen game state and board state every tick; metrics twice per second
Out	To M1	M1	Checked controls, joins, “Find me”, admin commands, board events; network metrics
Out	To phones	M5	Game state 30 times per second; join replies; ping; frame-rate limit
Out	To admin	M6	State 10 times per second, metrics, replies, board data
Out	To board	M7	Board state 10 to 30 times per second; config; vibrate commands

4. Main components

Component	What it does	Related modules
Web server and join	Serves the phone and admin pages, the QR code and the game code; announces the server on the local network	M5, M6, M7
Sessions	One session for each phone and the admin; player key for reconnecting within 30 s; one admin at a time	M1, M5, M6
Codec and checker	MessagePack encode and decode; checks type, fields and value ranges; a bad message gets an error reply	all
Input queue	Holds checked inputs until M1 reads them at the start of the next tick	M1
State broadcaster	Encodes the frozen state once and sends the same bytes to every phone	M1, M5, M6, M7
Delay and loss emulator	An up queue and a down queue for each player and for the board, set by the admin	M6
Ping and metrics	Ping twice per second, clock offset for the phone, ping, loss and frame-rate metrics	M1, M5, M6
Board link	Server side of the board connection: one TCP channel and one UDP stream	M7

5. How it works

Which link uses which protocol. This choice is important, so we made it message by message. A web browser can only open a WebSocket for two-way talk, so phones and the admin page use WebSocket. WebSocket runs on TCP, so every message arrives and the order is kept. The LED board is not a browser, so it can use both TCP and UDP. On TCP it gets the messages that must never be lost: the config (player colours, brightness, current limit) and the vibrate commands. The board also sends earthquake events and its status report on TCP. If one of these is lost, the game goes wrong, so TCP sends it again. The fast and repeated data goes on UDP: the board state, which comes 10 to 30 times per second, the sensor samples for the seismic graph, and the small delay probes. Here only the newest message is useful. If one UDP packet is lost, the next one comes 33 to 100 ms later and replaces it. TCP would wait for the lost packet and the LEDs would stop for a moment, so UDP is the better choice for this data. Each board state fits in one UDP packet (under 1 KB), so a packet is never split.

Joining. At the start the admin screen shows a QR code and a 6-digit game code. A player scans the QR code with the phone camera. The phone opens the game page from the main computer, so there is nothing to install. A player without a camera can open the short local address and type the code. The phone then chooses a team. The first time, the browser saves a random player key. If the page reloads or Wi-Fi drops, the phone sends the same key and gets the same motorcycle back within 30 seconds. The LED board finds the server by itself, because the server announces its name and port on the local network with mDNS.

Keeping the game state safe. Many phones send messages at the same time, but only M1 may change the game state, and only inside a tick. So M2 never writes to the state. A phone can only send controls; it can never send a position or a score. M2 decodes each message, checks its type and fields, and cuts every value into its range (for example steer is kept between -1 and 1). Each session can only control its own player, so one phone cannot move another motorcycle. Every message has an order number. A message with an old or repeated number is dropped. Good inputs wait in a queue, and M1 reads the queue at the start of the next tick. At the end of the tick M1 gives a frozen copy of the state. M2 encodes it once with MessagePack and sends the same bytes to all phones, so all players see exactly the same tick. The full game state is under 4 KB.

Ping, delay and loss. Twice per second M2 sends a ping with the server time. The phone answers with its own time, its frame rate and the number of the last state it got. From this M2 finds the round-trip time and the clock offset of the phone. Gaps

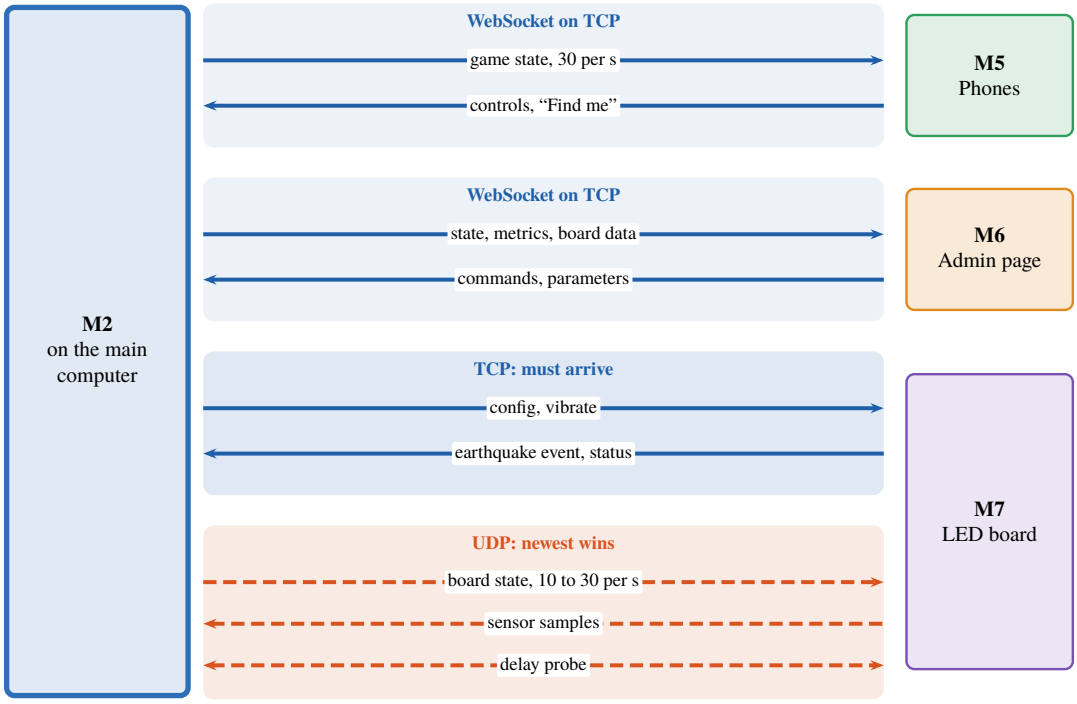

Figure 2: Every link and every message type. Blue solid arrows use TCP, orange dashed arrows use UDP.

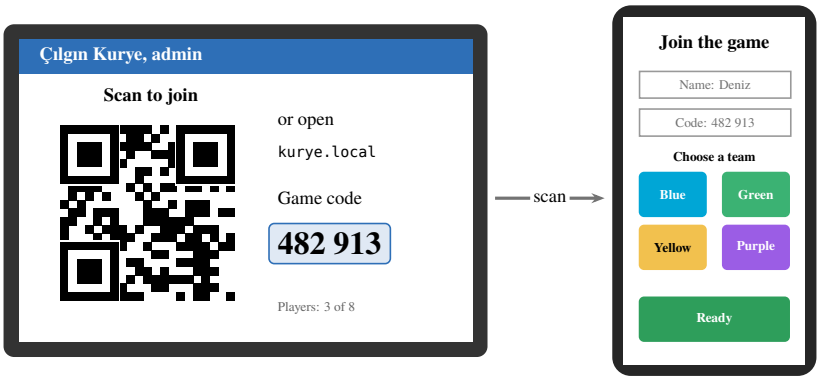

Figure 3: Joining: the QR code and game code on the admin screen, and the join page on a phone.

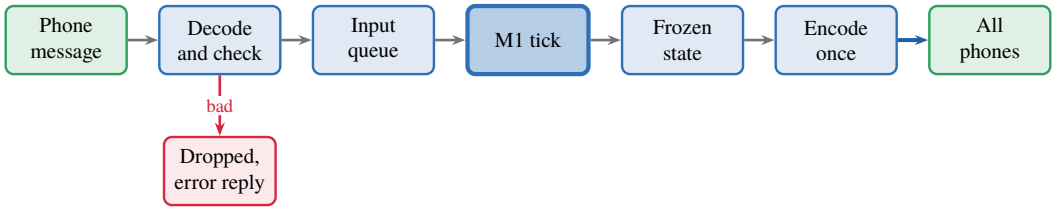

Figure 4: The path of an input. Only M1 writes the state, and everybody gets the same bytes.

in the order numbers give the packet loss. These values go to the admin graphs. For tests the admin can make the network worse for one player. Each player has an up queue and a down queue. A queue drops a message with the loss chance that the admin set, and holds the other messages for half of the added ping. The same tool works on the board link. The admin can also set a frame-rate limit, which M2 sends to the phone.

6. Testing and demonstration

M2 is tested with a fake M1 that sends a moving test state, with up to 8 scripted bot phones, and with the board emulator of M7. The tests check the message codec, bad and broken input, the load with 8 phones, how exact the added delay and loss are, reconnecting within 30 seconds, and the board link with 20% packet loss. In the demonstration phones join with the QR code or the game code. Then the admin changes one player's ping and loss live, and we watch the change on that phone and in the graphs.